GitHub Environment Setup for VPS Deploy

Oct 23, 2026 · @pinnacle

Overview

The **Deploy to VPS** workflow (`.github/workflows/deploy.yml`) deploys `apps/backend` to a Hostinger VPS. It needs **10 secrets** and **7 required variables** (plus 4 optional) in each GitHub environment, `production` and `staging`. Nothing is hardcoded in the YAML.

Each run does the following, in order. Every step checks first and only acts when something is missing:

1. Validates every secret and variable before touching the server.
2. Connects over SSH, accepting only the pinned host key.
3. Verifies server tools (Node, pnpm, pm2, Oracle client, nginx, certbot) and installs only what is missing.
4. Syncs the backend code with rsync.
5. Creates the Hostinger DNS record for your domain, if missing.
6. Creates the nginx site and SSL certificate, if missing.
7. Writes `.env` on the server from your secrets.
8. Runs `pnpm install`, Prisma generate and migrate, then reloads pm2.

Secrets vs variables

Use a **secret** for anything that causes damage if someone sees it; use a **variable** for everything else.

	Secret	Variable
Visible after saving	No, write-only (you can only replace it)	Yes, shown in Settings
Shown in workflow logs	No, masked as `***`	Yes, plain text
Used in YAML as	`${{ secrets.NAME }}`	`${{ vars.NAME }}`

	Secret	Variable
Use for	Passwords, keys, tokens, server details	Domains, ports, paths, usernames

The four server details (`REMOTE_HOST` , `REMOTE_USER` , `REMOTE_TARGET` , `PM2_APP_NAME`) are not passwords. They are secrets only to keep them out of the logs. Names must match exactly, including upper case.

Create the GitHub environments

Create two environments named exactly `production` and `staging` ; each holds its own copy of every secret and variable. The workflow picks the set matching the environment you choose when you run it.

1. Open your repository on GitHub and go to **Settings** → **Environments**.
2. Click **New environment**, type `production` , and click **Configure environment**.
3. Under **Deployment protection rules**, tick **Required reviewers** and add yourself (and anyone who must approve production deploys). Click **Save protection rules**.
4. Under **Deployment branches and tags**, choose **Selected branches and tags**, click **Add deployment branch or tag rule**, and enter `main` .
5. Under **Environment secrets**, click **Add environment secret** once for each secret in the next section.
6. Under **Environment variables**, click **Add environment variable** once for each variable in the section after that.
7. Repeat steps 2 to 6 for `staging` , using the staging server, domain and passwords. Reviewers are optional for staging.

Add secrets and variables inside the environment, not under repository-level **Secrets and variables → Actions**. Repository-level values would be shared by production and staging.

Secrets (10, all required)

Add each under **Environment secrets**. Paste values exactly, with no surrounding quotes.

Name	What it is	Example	Where to get it
`SSH_PRIVATE_KEY`	Private key GitHub uses to log in to the VPS	`-----BEGIN OPENSSH PRIVATE KEY-----` … whole file	Generate it (see the setup section)

Name	What it is	Example	Where to get it
`SSH_KNOWN_HOSTS`	The VPS host keys, so GitHub connects only to your real server	`203.0.113.10 ssh-ed25519 AAAAC3Nz…` (one line per key)	Read on the server (see the setup section)
`HOSTINGER_API_TOKEN`	Token to create the DNS record	A long random string	hPanel → Profile → API
`REMOTE_HOST`	VPS IP address or hostname	`203.0.113.10`	hPanel → VPS → Overview
`REMOTE_USER`	Linux user the deploy logs in as	`deploy` or `root`	The user that owns your app and has sudo
`REMOTE_TARGET`	Absolute app root folder on the server	`/var/www/myapp`	Your choice; code goes to `<this>/apps/backend`
`PM2_APP_NAME`	pm2 process name	`myapp-backend`	Letters, digits, `.` `_` `-` only
`DATABASE_URL`	Prisma database connection string	`mysql://user:pass@localhost:3306/mydb`	Your database settings
`DB_PASSWORD`	Oracle database password	`S3cure!Pass`	Your Oracle DBA
`MAIL_PASSWORD`	Mail account or app password	`abcd efgh ijkl mnop`	Your mail provider (e.g. Gmail app password)

Passwords may contain any characters; the workflow escapes quotes and backslashes when it writes `.env`. Avoid `$` in passwords Prisma reads, because Prisma may expand it as a variable.

Variables (7 required, 4 optional)

Add each under **Environment variables**. Domains must be lowercase.

Name	Required	What it is	Example
`DOMAIN`	Yes	Full domain the app is served on	`api.example.com`
`ROOT_DOMAIN`	Yes	Domain you own in Hostinger; `DOMAIN` must equal it or end with it	`example.com`
`PORT`	Yes	Port your Node app listens on (1024–65535)	`4000`
`SSL_EMAIL`	Yes	Email for Let's Encrypt expiry notices	`admin@example.com`
`DB_USER`	Yes	Oracle database user	`APP_USER`

Name	Required	What it is	Example
`DB_CONNECT_STRING`	Yes	Oracle connect string	`dbhost:1521/ORCLPDB1`
`MAIL_USER`	Yes	Mail account the app sends from	`noreply@example.com`
`ORACLE_CLIENT_PATH`	No	Oracle Instant Client folder; if empty, the client is auto-detected, or installed to `/opt/oracle/instantclient`	`/opt/oracle/instantclient_21_12`
`SERVER_IP`	No	VPS IPv4 for the DNS record; needed only if `REMOTE_HOST` is a hostname	`203.0.113.10`
`NODE_VERSION`	No	Node major to install, used only if no Node exists on the server	`22`
`AUTO_INSTALL`	No	`false` = only verify tools and stop if one is missing; `true` (default) = install missing tools	`false`

For staging, use a different `DOMAIN` (for example `staging-api.example.com`) with the same `ROOT_DOMAIN`.

Generate the keys and token

SSH_PRIVATE_KEY

Create a key used only for GitHub deploys. Run this on your own computer:

```
ssh-keygen -t ed25519 -C "github-actions-deploy" -f gh_deploy -N ""
```

1. Copy the whole content of `gh_deploy` (including the `BEGIN` and `END` lines) into the `SSH_PRIVATE_KEY` secret.
2. Log in to the VPS as `REMOTE_USER` and add the public half:

```
mkdir -p ~/.ssh && chmod 700 ~/.ssh
echo "PASTE THE ONE LINE FROM gh_deploy.pub HERE" >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
```

3. Delete `gh_deploy` from your computer once it is saved in GitHub.

SSH_KNOWN_HOSTS

Read the host keys on the server itself, so nobody can fake them. Open hPanel → **VPS** → **Browser terminal** and run, with your VPS IP in place of `203.0.113.10`:

```
for k in /etc/ssh/ssh_host_*_key.pub; do echo "203.0.113.10 $(cut -d' ' -f1,2
"$k")"; done
```

Paste all the printed lines into the `SSH_KNOWN_HOSTS` secret. The IP or hostname at the start of each line must match `REMOTE_HOST` exactly.

HOSTINGER_API_TOKEN

1. In hPanel, open your **Profile** → **API**.
2. Create a new token, give it a name such as `github-deploy`, and set an expiry date.
3. Copy it immediately into the `HOSTINGER_API_TOKEN` secret; it is shown only once.

This token can manage your whole Hostinger account, so keep **Required reviewers** on for production and rotate it before it expires.

Server prerequisites

Four things must be true on the server and in Hostinger before the first run; the workflow handles everything else.

- [ ] **Deploy user:** `REMOTE_USER` is `root` or a user with passwordless sudo. Check with `sudo -n true && echo OK` as that user. To grant it (replace `deploy`): `echo 'deploy ALL=(ALL) NOPASSWD:ALL' | sudo tee /etc/sudoers.d/deploy && sudo chmod 440 /etc/sudoers.d/deploy`.
- [ ] **Same user as your existing Node:** if Node, pnpm or pm2 is installed through nvm for a specific user, set `REMOTE_USER` to that user so the existing install is reused.
- [ ] **Domain in Hostinger:** `ROOT_DOMAIN` is registered in your Hostinger account and uses Hostinger nameservers. The workflow creates the DNS record but never buys a domain.
- [ ] **Ports open:** hPanel → **VPS** → **Firewall** allows TCP 22 (SSH), 80 (HTTP) and 443 (HTTPS).

If your server already runs Apache, LiteSpeed or Caddy, nginx is not installed and the nginx/SSL step is skipped; point that web server to `PORT` yourself.

First run and verification

Run the first deploy with `AUTO_INSTALL=false` to confirm what the server already has before anything is installed.

1. In the `production` environment, add the variable `AUTO_INSTALL` = `false`.
2. Go to the repository's **Actions** tab → **Deploy to VPS** → **Run workflow**, choose `production`, and click **Run workflow**.
3. Approve the run when GitHub asks (this is the Required reviewers rule).
4. Open the **Verify server tools** step and read each line:

```
    [ok]      node v20.11.0 at /home/deploy/.nvm/...  -> already installed,
skipped
    [ok]      Oracle client at /opt/oracle/instantclient_21_12  -> already
installed, skipped
    [ok]      nginx 1.24.0 (existing sites untouched)  -> already installed,
skipped
==> SUMMARY: everything was already installed - nothing installed or changed.
```

5. If a line shows an error for a missing tool, either install it yourself or change `AUTO_INSTALL` to `true` and run again.
6. Check that https:// `DOMAIN` opens your app with a padlock. The first SSL run waits up to 10 minutes for new DNS to resolve.

After the first success, later runs only sync code, rewrite `.env`, install app packages, migrate and reload pm2; the DNS, site and SSL steps report "already exists - skipping".

Troubleshooting

Most failures come from a name typo, a value in the wrong environment, or a key that does not match the server.

Error in the log	Cause	Fix
`SSH_KNOWN_HOSTS secret is empty` / `... is invalid`	Secret missing, misspelled, or added at repository level	Add it under the environment, with the exact name

Error in the log	Cause	Fix
`Host key verification failed`	`SSH_KNOWN_HOSTS` lines don't match `REMOTE_HOST`, or the VPS was rebuilt	Re-run the host-key command and replace the secret
`Permission denied (publickey)`	Public key not in `authorized_keys` of `REMOTE_USER`	Add the `gh_deploy.pub` line for that user
`Installing X needs root`	`REMOTE_USER` has no passwordless sudo	Grant sudo (prerequisites) or install X manually
`X is missing on the server and AUTO_INSTALL=false`	Tool not found and installs are off	Install X yourself, or set `AUTO_INSTALL` = `true`
`Node exists at ... but is not usable by deploy`	Node belongs to another user	Set `REMOTE_USER` to that user
`ORACLE_CLIENT_PATH is '...' but the client is at '...'` (warning)	Variable points to the wrong folder	Set the variable to the folder shown
`DOMAIN must be ROOT_DOMAIN or a subdomain of it`	Domains don't match, or have upper case	Use lowercase; `DOMAIN` must end with `ROOT_DOMAIN`
`Set vars.SERVER_IP to the VPS IPv4 address`	`REMOTE_HOST` is a hostname	Add `SERVER_IP` with the VPS IPv4
curl `401` on the DNS step	Hostinger token wrong or expired	Create a new token and replace the secret
certbot fails after waiting	DNS not pointing to the VPS yet, or port 80 closed	Check the A record and firewall, then run again
`pnpm install` lockfile error	`pnpm-lock.yaml` out of date	Run `pnpm install` locally and commit the lockfile